

Proposta Executiva Solução Big Data para uma Frota Autônoma Conectada

Arquitetura de dados resiliente, escalável e em tempo real — teoricamente fundamentada e voltada a valor de negócio.

FOLHA DE ROSTO • COMPONENTES DO GRUPO (NOME COMPLETO + DRE)

Izabela Lima da Silva

DRE 124156557

Caio Meirelles

DRE 122071557

Esta proposta especifica uma arquitetura de **Big Data** para monitorar, em tempo real, uma frota de veículos autônomos e conectados de um consórcio de seis locadoras. Cada escolha é **fundamentada na literatura** da disciplina (21 referências primárias) e conectada ao **valor de negócio** que entrega. A tese central: o esquema estrela em PostgreSQL da Avaliação 02 deixa de ser um DW isolado e passa a ser a **camada Gold** de um Lakehouse orientado a fluxo contínuo.

DIVISÃO DO TRABALHO (DEFESA ORAL)

Parte A — Infraestrutura & fluxo (borda, ingestão Kafka, Kappa, processamento Spark/Flink, conectividade): seções 7–12. Parte B — Armazenamento, interface & IA (persistência poliglota, dashboard, cobrança, concierge): seções 13–15. Emergências (hot path) é a ponte.

Sumário

1. Contexto e definição do problema

2. Valor para o cliente

3. Da Avaliação 02 (DW) ao Big Data e a extensão do modelo

4. Fundamentos teóricos (4 partes da ementa)

5. Objetivos, escopo e requisitos

6. Visão geral da arquitetura (pipeline)

7. Camada de borda (Edge Computing)

8. Camada de ingestão (Kafka)

9. Lambda vs. Kappa e o Lakehouse

10. Lakehouse: Delta Lake

11. Camada de processamento (Spark/Flink)

12. Persistência poliglota (NoSQL)

13. Serving: dashboard, cobrança e emergências

14. Concierge de viagem por IA (RAG)

15. Casos de uso e rastreabilidade das entidades

16. Custos, nuvem e prova de conceito

17. Alternativas descartadas

18. Conclusão

19. Referências

01 Contexto e definição do problema

O consórcio de seis locadoras decidiu **automatizar a operação**: pelo aplicativo, o cliente reserva e indica o local; o **veículo sai sozinho do pátio**, navega no trânsito e estaciona no ponto pedido; ao fim, retorna para higienização e manutenção. Cada veículo tem sensores internos (motor, chassi) e externos (telemetria inercial, câmeras 360°, impacto) e um **computador de bordo** (Edge), gerando um fluxo contínuo de dados.

O problema é sustentar quatro necessidades — (1) **acompanhamento** em dashboard; (2) **reserva** eficiente; (3) **cobrança automática** pós-uso; (4) **apoio a emergências** com dossiê regulatório — além

do *extra* do **concierge por IA**.

RESTRIÇÃO-CHAVE

Banda móvel é cara e finita: inviável transmitir vídeo bruto contínuo. O sistema deve suportar alta concorrência sem perda de pacotes, com **ordenação estrita por veículo e timestamp**, mesmo sob estresse de carga.

02 Valor para o cliente

A arquitetura é meio; o valor é o fim. Cada capacidade técnica move um indicador de negócio do consórcio:

UTILIZAÇÃO

Menos veículos ociosos

Redistribuição inteligente (Markov + rotas em tempo real) maximiza o giro dos ativos.

↑ taxa de utilização

RECEITA

Cobrança dinâmica justa

Captura acréscimos por km, consumo e infrações; transparência reduz contestações.

↑ receita / locação

CUSTO

Sem balcão nem entrega

Reserva pelo app + veículo autônomo cortam atendimento e logística.

↓ custo / locação

SINISTROS

Resposta e regulação rápidas

Deteção na borda + dossiê automático aceleram socorro e seguradora.

↓ tempo de resposta

DISPONIBILIDADE

Manutenção preditiva

Antecipar panes reduz o veículo parado e a quebra em operação.

↓ downtime

EXPERIÊNCIA

Concierge por IA

Roteiro personalizado por voz diferencia o serviço e fideliza.

↑ satisfação / NPS

03 Da Avaliação 02 (DW) ao Big Data e a extensão do modelo

Na Avaliação 02 modelamos um **Data Warehouse** dimensional em **PostgreSQL** (schema `dw_locadora`), integrando **4 fontes OLTP** e carregando três fatos — **Reserva, Locação, Movimentação** — sobre cinco dimensões conformadas (Tempo, Cliente, Veículo, Empresa, Pátio *role-playing*), formando uma *fact constellation*. O ETL, em cinco etapas, gera relatórios e a **previsão de ocupação de pátios por cadeias de Markov**. A separação OLAP/OLTP é o axioma fundador do DW — o paper cita explicitamente *fleet management* como caso clássico (Chaudhuri & Dayal, 1997).

Esse modelo, porém, é **batch (D-1)** e não cobre a frota conectada. A modernização segue a ementa: em vez de manter DW e data lake separados — que sofrem de *staleness* e alto TCO — adotamos o **Lakehouse** (Armbrust et al., 2021): o esquema estrela vira a camada **Gold** sobre Delta/MinIO, alimentada por ELT do streaming.

EXTENSÃO DIMENSIONAL (AVALIAÇÃO 03)

Estendemos o modelo com quatro novos fatos — **Fato_Telemetria_Diaria**, **Fato_Cobranca**, **Fato_Sinistro**, **Fato_Manutencao** — e novas dimensões (**Dim_Tempo_Detalhe** de grão minuto, **Dim_Sensor** SCD-2, **Dim_TipoEvento**, **Dim_FaixaConducao**), reusando as dimensões conformadas. O DDL (`06_ddl_extensao.sql`) foi validado executando `01 + 06` num PostgreSQL real. As tabelas são carregadas por ELT do Silver; a matriz de Markov é estendida para condicional (por faixa horária/categoria) e incremental.

04 Fundamentos teóricos

Os fundamentos organizam-se nas quatro partes da ementa; cada princípio ancora uma decisão de arquitetura.

Parte I — VLDB e computação distribuída

Em clusters commodity, **falhas são a norma**: monitoramento e recuperação automática são integrais ao projeto (Ghemawat et al., 2003) — justificando réplicas (Kafka/Cassandra) e containers reconstituíveis. O princípio da **localidade** — mover a computação para perto do dado (Dean & Ghemawat, 2004) — fundamenta a computação de borda que pré-agrega no veículo. A disciplina temporal culmina no **TrueTime** do Spanner e na *consistência externa* (Corbett et al., 2012), alicerce da exigência de ordenação estrita por timestamp; medir o SLA no percentil 99,9 e não na média (DeCandia et al., 2007) orienta as metas do apoio a emergências.

Parte II — Data Warehouse e BI

O DW é *subject-oriented, integrated, time-varying, non-volatile* e fisicamente separado do OLTP (Chaudhuri & Dayal, 1997); a generalização N-dimensional do agregado é o operador **CUBE** com o valor `ALL` (Gray et al., 1997), base do cubo executivo frota × tempo × pátio × empresa. A distinção entre agregados distributivos/algébricos e holísticos governa a divisão hot-path/batch. O motor da Gold é o **Spark SQL** com o otimizador Catalyst (Armbrust et al., 2015); o **log de transações do Delta** dá ACID e time travel ao object store (Armbrust et al., 2020).

Parte III — Big Data, streaming e engenharia de IA

O **MapReduce** restringe o modelo a map/reduce para paralelizar e tolerar falhas por re-execução determinística (Dean & Ghemawat, 2004); o *combiner* fundamenta a pré-agregação na borda. O **RDD** do Spark tolera falhas por **linhagem**, com reuso em memória até 20× (Zaharia et al., 2012) — núcleo do cálculo iterativo da Markov. A ingestão gira no **Kafka** como log particionado, cuja ordenação estrita por partição atende a ordem por veículo sem coordenação global (Kreps et al., 2011). O Spark Streaming garante **exactly-once** por micro-batches idempotentes (Zaharia et al., 2013); o

Flink oferece dataflow único com **event-time e watermarks** e snapshots ABS (Chandy-Lamport) (Carbone et al., 2015). O *survey* situa a stack na 2ª geração e justifica **Kappa sobre Lambda** (Fragkoulis et al., 2020). Na IA, o **RAG** ancora a geração em conhecimento recuperável, reduz alucinação e dá proveniência (Lewis et al., 2020).

Parte IV — Persistência poliglota, NoSQL, NewSQL, vetoriais

A raiz é o **teorema CAP**: sob partição, consistência e disponibilidade não coexistem (Gilbert & Lynch, 2002); segmenta-se o sistema por trade-off (Cattell, 2011). A telemetria herda de **Dynamo** (always-writeable, quórums N/R/W, vector clocks) (DeCandia et al., 2007); o **Cassandra** entrega alta escrita via LSM e *clustering* por timestamp que materializa a ordem (Lakshman & Malik, 2009). A memória do concierge usa **bancos vetoriais** (busca por similaridade, consistência delta) (Guo et al., 2022); no polo consistente, o **NewSQL** mostra SQL+ACID escalando, posicionando o PostgreSQL como o lugar da cobrança e da modelagem dimensional.

05 Objetivos, escopo e requisitos

Objetivos funcionais

- **Acompanhamento (dashboard)**: frota em tempo real, emergências e financeiro.
- **Reserva** mais objetiva; **cobrança automática** pós-devolução com ajustes por uso.
- **Apoio a emergências** e dossiê para regulação; *extra*: **concierge por IA (voz)**.

Requisitos não funcionais (os cinco desafios)

#	Requisito	Como a arquitetura atende (fundamento)
1	Banda finita	Pré-agregação na borda; só agregados/exceções trafegam (Dean & Ghemawat, 2004).
2	Alta concorrência	Kafka como log durável absorve picos; sem perda de pacotes (Kreps, 2011).
3	Ordenação estrita	Partição por <code>vehicle_id</code> + event-time/watermarks (Carbone, 2015; Corbett, 2012).
4	Resiliência	Réplicas, linhagem e replay (Ghemawat, 2003; Zaharia, 2012).
5	ACID lote↔stream	Delta Lake unifica sobre as mesmas tabelas (Armbrust, 2020).

06 Visão geral da arquitetura PARTE A

O pipeline cobre **ingestão** → **processamento** → **armazenamento** → **visualização**, com três caminhos de latências distintas.

```
# Pipeline fim a fim
[Borda/Edge (Avro)] > [MQTT > ponte > Kafka (telemetry.raw | alerts.critical)] >
├─ HOT colisão → Spark Streaming → despacho + dossiê
├─ WARM telemetria 1s → Flink (event-time/watermarks) → rotas, score
└─ COLD Delta Bronze → Spark batch → km, manutenção, Markov
[Delta Lake Bronze→Silver→Gold (DW estendido)] > [NoSQL: Cassandra · Mongo · Redis]
> [Streamlit · Cobrança · Concierge]
```

07 Camada de borda (Edge Computing) PARTE A

Como a banda é escassa, o computador de bordo pré-processa e divide os dados em dois fluxos — o *combiner* do MapReduce aplicado à borda (Dean & Ghemawat, 2004).

WARM PATH

Rotina agregada

Telemetria (velocidade, GPS, bateria, sensores) agregada e enviada a cada 1 s em **Avro** (binário) sobre MQTT — substitui o JSON textual. A borda faz downsampling e extração de features.

HOT PATH

Alerta imediato

Deteção de colisão **na borda** (reação sub-segundo); prioriza a rede e envia payload de emergência com ring buffer de ~30 s. Orientado a evento.

VALOR

Corta o maior custo recorrente (egress) e viabiliza a resposta que não pode esperar a nuvem; store-and-forward garante zero perda na desconexão.

08 Camada de ingestão: Kafka PARTE A

Barramento orientado a eventos que desacopla frota (produtores) de processadores (consumidores) — um **log particionado** com consumo pull por offset e retenção (Kreps et al., 2011). Um broker MQTT + Kafka Connect faz a ponte borda→Kafka.

- **Tópicos isolados:** `telemetry.raw` vs `alerts.critical` — o alarme nunca fica preso atrás da telemetria.
- **Partição por** `vehicle_id` → ordem estrita por veículo, sem coordenação global.
- **Replicação + ISR** → tolerância a falhas; **at-least-once** + dedup por chave (aplicação); **Avro + Schema Registry**.

09 Lambda vs. Kappa e o Lakehouse PARTE A

Critério	Lambda	Kappa
Base de código	Dupla	Única
Consistência	Risco de divergência	Uma verdade
Reprocessar histórico	Batch nativo	Replay + Lakehouse

Decisão: o problema é *streaming-first*; o stream durável do Kafka unifica os caminhos e elimina a dupla implementação (Fragkoulis et al., 2020). Adotamos **Kappa**, cobrindo o lote pesado com o Lakehouse.

10 Lakehouse: Delta Lake PARTE A

O object store é key-value sem atomicidade cross-key; o **log de transações do Delta** traz ACID serializável, *data skipping*, time travel e MERGE/DELETE (Armbrust et al., 2020, 2021). Organização **medallion**: Bronze (bruto, append-only) → Silver (limpo, conformado) → **Gold** (o DW estrela estendido). Executa em Spark sobre MinIO.

ACID ENTRE BATCH E STREAMING

Batch e streaming operam sobre as mesmas tabelas — relatórios e visão ao vivo leem a mesma verdade; o time travel monta dossiês reproduzíveis.

11 Camada de processamento (Spark/Flink)

PARTE A

BATCH · HORAS Apache Spark RDD com tolerância por linhagem (Zaharia, 2012); consolida do Delta Bronze; manutenção preditiva e Markov; generaliza o MapReduce.	STREAM · SEGUNDOS Spark Streaming Hot path das emergências; micro-batches com exactly-once idempotente (Zaharia, 2013) — evita cobrar em dobro; orquestra a resposta na nuvem.	STREAM · MS Apache Flink Dataflow único; event-time + watermarks e ABS (Carbone, 2015); rotas, congestão e score de condução.
--	---	--

Três regimes, três SLAs — a reação sub-segundo à colisão é embarcada; a nuvem orquestra a resposta.

12 Persistência poliglota (NoSQL)

PARTE B

A escolha segue o padrão de acesso e o **teorema CAP** (Gilbert & Lynch, 2002): segmentar e atribuir a cada subserviço seu trade-off (Cattell, 2011).

Tecnologia	Modelo	Uso / fundamento	Trade-off
Cassandra	Wide-column (AP)	Telemetria: escrita massiva (LSM), clustering por timestamp materializa a ordem (Lakshman, 2009; DeCandia, 2007).	Consulta ad-hoc limitada.
MongoDB	Documento	Cadastrais, dossiês de sinistro, relatórios; esquema flexível.	Não para série temporal pura.
Redis	Key-value (RAM)	Cache quente: estado ao vivo, rotas, geoespacial (sub-ms).	Volátil; não é fonte da verdade.
PostgreSQL / Delta	Relacional (CP)	Cobrança (ACID), DW/Gold, treino de ML — polo consistente (NewSQL-like).	Latência maior que caches.

Mapeamento CAP: telemetria (AP, always-writeable) vs. cobrança/dossiê (CP) — o mesmo trade-off carrinho vs. checkout do Dynamo.

13 Serving: dashboard, cobrança e emergências PARTE B

Dashboard (Streamlit)

OLAP sobre a **Gold**, nunca sobre os OLTP (Chaudhuri & Dayal, 1997); ao vivo via Redis pub/sub + auto-refresh. Mapa geoespacial, cartões (veículos ativos, bateria média, alertas), triagem que pisca em vermelho, e painéis de reservas, financeiro, viagens e oficina. O cubo frota × tempo × pátio × empresa apoia a visão executiva (Gray et al., 1997).

Motor de cobrança dinâmica

Acionado na devolução; valor final = base ± ajustes (km, tempo, consumo, infrações, score do Flink). *Exactly-once* idempotente evita duplicidade; auditável por time travel — **valor: ↑ receita, ↓ contestações.**

Apoio a emergências (hot path)

Borda detecta → `alerts.critical` → Spark orquestra → resposta (guincho, seguradora, substituto). Em paralelo monta-se o **dossiê de sinistro** (sensores + vídeo + timeline reconstruída) com todas as informações à regulação — **valor: ↓ tempo de resposta e de regulação.**

14 Concierge de viagem por IA (RAG) PARTE B

Backend de assistente de voz que combina **PLN** (intenção/entidades) e *function calling*. O **RAG** ancora a geração em conhecimento recuperável (índice denso), reduz alucinação e dá proveniência (Lewis et al., 2020) — sem chave de LLM paga.

[Voz] > ASR > [PLN + agente + function calling] > banco vetorial (RAG) > POIs + agenda > TTS + GPS

EXEMPLO

"Após a reunião, quero almoçar em restaurante típico bem avaliado com estacionamento fácil e, à tarde, ver arte nos meus gostos." → o agente lê a localização, consulta POIs por similaridade e a agenda, monta o roteiro e **injeta os waypoints no GPS**. A mesma base de recuperação citável sustenta os dossiês de emergência. Privacidade: consentimento, PII, wake-word on-device.

15 Casos de uso e rastreabilidade das entidades

Para demonstrar que a modelagem não é abstrata, três casos reais — **crítico, usual e analítico** — mostram onde cada fato e dimensão aparece e como o produto entrega valor. Dois deles apontam

explicitamente para a **extensão do modelo**.

Caso 1 — Colisão em túnel (crítico, resposta rápida)

Às 18h23, o veículo autônomo colide dentro de um túnel (zona de sombra). O sensor de impacto detecta a desaceleração violenta **na borda** (sub-segundo) e dispara um *payload* de emergência (ring buffer de ~30 s) em `alerts.critical` ; ao sair do túnel, a telemetria pré-impacto retida em *store-and-forward* chega atrasada e o Flink a reordena por *event-time/watermark*. O Spark Streaming orquestra guincho, seguradora e a reserva de um substituto. **Aplicações múltiplas:** resposta a emergência; **dossiê de regulação** — o *time travel* do Delta com a `Dim_Sensor` (SCD-2) reconstrói qual firmware do sensor estava ativo no minuto do impacto; reserva de substituto (disponibilidade no Redis + `Fato_Reserva`); e recálculo da locação interrompida (`Fato_Cobranca`). **Entidades:** `Fato_Sinistro` , `Dim_TipoEvento` , `Dim_Sensor` , `Dim_Tempo_Detalhe` , `Dim_Veiculo` , `Dim_Patio` , `Dim_Cliente` . **Aponta modificação:** a `Dim_Tempo` (grão dia) não situa o evento no minuto → justifica a `Dim_Tempo_Detalhe` ; sem a `Dim_Sensor` SCD-2 não há dossiê *point-in-time*. *Valor:* ↓ *tempo de resposta e de regulação*.

Caso 2 — Devolução com condução agressiva (usual, financeiro)

O cliente devolve o veículo após três dias, com excesso de velocidade recorrente e uma multa capturada. Em *warm path*, o Flink calculou continuamente o score de condução (janelas + estado por veículo) → `Dim_FaixaConducao` = "Agressivo" (com `fator_tarifa`). Na devolução, o evento em `billing.events` aciona o motor de cobrança (*exactly-once*), que consolida a `Fato_Telemetria_Diaria` (km, velocidade, eventos bruscos) e a infração e calcula a `Fato_Cobranca` = base ± acréscimos. **Aplicações múltiplas:** cobrança justa e automática; transparência ao cliente (dossiê da viagem citável via RAG); perfil de risco alimentando a política tarifária futura; e contestação reprocessável por *time travel*. **Aponta modificação:** o DW da Avaliação 02 não tinha **nenhuma medida financeira** nos fatos → a `Fato_Cobranca` preenche essa lacuna, e o score derivado vira a dimensão `Dim_FaixaConducao` . *Valor:* ↑ *receita por locação*; ↓ *contestações*.

Caso 3 — Pico no aeroporto, pane iminente e concierge (analítico, IA)

Sexta, 17h: a demanda concentra-se no Galeão. A **matriz de Markov condicional** (agora por faixa horária, via `Dim_Tempo_Detalhe`) prevê a migração de veículos e dispara o **reposicionamento** de veículos autônomos vazios (`Fato_Movimentacao` + `Dim_Patio` origem/destino). Em paralelo, o batch Spark sobre a `Fato_Telemetria_Diaria` calcula a `Fato_Manutencao` (probabilidade de falha alta num veículo) → agenda manutenção e o retira do *pool* (Redis). E o concierge (PLN + RAG sobre banco vetorial de POIs) monta o roteiro do cliente premium a bordo, injetando *waypoints* no GPS. **Aplicações múltiplas:** reposicionamento preditivo; manutenção preditiva; concierge; e a visão executiva (cubo frota × tempo × pátio × empresa). **Aponta modificação:** a Markov incondicional torna-se condicional (exige a `Dim_Tempo_Detalhe`) e a manutenção preditiva exige os novos `Fato_Manutencao` e `Fato_Telemetria_Diaria` . *Valor:* ↑ *utilização*; ↓ *downtime*; ↑ *NPS*.

Rastreabilidade: cada entidade modelada em ação

Entidade (+ = extensão)	1 · Colisão (crítico)	2 · Cobrança (usual)	3 · Frota (analítico)
Fato_Sinistro +	dossiê + resposta	—	—

Entidade (+ = extensão)	1 · Colisão (crítico)	2 · Cobrança (usual)	3 · Frota (analítico)
Fato_Cobranca +	recálculo da locação	valor ± acréscimos	—
Fato_Telemetria_Diaria +	buffer pré-impacto	km, velocidade, eventos	base do score / manutenção
Fato_Manutencao +	—	—	prob. de falha → agenda
Fato_Movimentacao	despacho ao pátio	—	Markov → reposicionamento
Fato_Reserva / Locação	veículo substituto	locação faturada	pool de disponíveis
Dim_Sensor + (SCD-2)	firmware na hora do sinistro	—	saúde do sensor
Dim_TipoEvento +	colisão	violação de trânsito	pane / superaquecimento
Dim_FaixaConducao +	—	Agressivo → tarifa	perfil por veículo
Dim_Tempo_Detalhe +	minuto 18:23	janelas da viagem	faixa horária / pico
Dim_Patio · Veículo · Cliente	pátio próximo · quem	retirada / devolução	origem / destino

16 Custos, nuvem e prova de conceito

BANDA / EGRESS

Filtrar na borda

Não enviar vídeo bruto corta o maior custo recorrente.

COMPUTE

Elasticidade

Autoscaling + spot/preemptível; clusters efêmeros no batch.

STORAGE

Tiering

Quente (Redis/Cassandra) vs. frio (Delta/MinIO), TTL, Bronze→Gold.

OPERAÇÃO

Gerenciado vs. próprio

Serviços gerenciados reduzem custo; trade-off com lock-in.

PROVA DE CONCEITO EXECUTÁVEL

O sistema foi implementado em **docker-compose** (Kafka, Flink, Spark, Cassandra, MongoDB, Redis, MinIO, PostgreSQL Gold, Streamlit), com **115 testes unitários** verdes e o DDL do DW estendido validado. Free tiers para experimentação: Dataproc/Flink Playground, Databricks, MongoDB Atlas, Astra DB, Redis Cloud, DynamoDB.

17 Alternativas descartadas

Opção descartada	Por que não
Somente Data Warehouse	Batch e schema-on-write não dão tempo real nem absorvem sensores de alto volume.
Somente batch	Emergências e roteirização exigem segundos/milissegundos.
Arquitetura Lambda	Base de código dupla e risco de divergência; problema é streaming-first.
Banco relacional único	Não escala escrita massiva nem entrega cache sub-ms (CAP).
Vídeo bruto pela rede	Banda cara e finita; fica na borda.
Hadoop MapReduce clássico	Alta latência e I/O em disco; o Spark (DAG, in-memory) o substitui no mesmo papel batch.

18 Conclusão

A solução é uma arquitetura **resiliente, escalável e em tempo real, teoricamente fundamentada** (21 referências primárias) e orientada a **valor**: filtra na borda (banda), isola fluxos no Kafka (ordem por veículo), adota **Kappa + Delta Lake** (ACID lote↔stream), processa em três regimes (Spark/Flink), persiste de forma poliglota (CAP) e serve dashboard, cobrança auditável, dossiês e concierge RAG. Ela **envolve e estende o DW** da Avaliação 02 como camada Gold e foi materializada num **sistema executável validado**, mantendo o custo de nuvem sob controle e movendo os indicadores de negócio do consórcio.

19 Referências

Armbrust, M. et al. (2015). Spark SQL: Relational Data Processing in Spark. SIGMOD.

Armbrust, M. et al. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. PVLDB.

Armbrust, M. et al. (2021). Lakehouse: A New Generation of Open Platforms. CIDR.

Carbone, P. et al. (2015). Apache Flink: Stream and Batch Processing in a Single Engine. IEEE Data Eng. Bull.; e Lightweight Asynchronous Snapshots for Distributed Dataflows. arXiv:1506.08603.

Cattell, R. (2011). Scalable SQL and NoSQL Data Stores. SIGMOD Record.

Chang, F. et al. (2006). Bigtable: A Distributed Storage System for Structured Data. OSDI.

Chaudhuri, S. & Dayal, U. (1997). An Overview of Data Warehousing and OLAP Technology. SIGMOD Record.

Corbett, J. C. et al. (2012). Spanner: Google's Globally-Distributed Database. OSDI.

Dean, J. & Ghemawat, S. (2004). MapReduce: Simplified Data Processing on Large Clusters. OSDI.

DeCandia, G. et al. (2007). Dynamo: Amazon's Highly Available Key-value Store. SOSP.

Fragkoulis, M. et al. (2020). A Survey on the Evolution of Stream Processing Systems. arXiv:2008.00842.

Ghemawat, S., Gobioff, H. & Leung, S.-T. (2003). The Google File System. SOSP.

Gilbert, S. & Lynch, N. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News.

Gray, J. et al. (1997). Data Cube: A Relational Aggregation Operator. Data Mining and Knowledge Discovery.

Guo, R. et al. (2022). Manu: A Cloud Native Vector Database Management System. PVLDB.

Kreps, J., Narkhede, N. & Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. NetDB.

Lakshman, A. & Malik, P. (2009). Cassandra: A Decentralized Structured Storage System. LADIS/ACM SIGOPS.

Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

Zaharia, M. et al. (2012). Resilient Distributed Datasets. NSDI.

Zaharia, M. et al. (2013). Discretized Streams. SOSP.